

Web API - Einführung



1

Ein bisschen Theorie.

Aber die Schulung soll sehr praktisch sein,
weshalb Theorie nur "angeschnitten" wird.

Für tiefe konzeptionelle Details ist das Studium da.

Theorie - RESTful Services

Was ist REST?

- Representational State Transfer: REST, seltener ReST
- Ein Paradigma für die Softwarearchitektur von verteilten Systemen
- insbesondere Webservices: RESTful API Web services
- Gut geeignet für **CRUD** Operationen: Create, Read, Update, Delete
- Schicht 4 des TCP/IP Referenzmodells: Anwendungsschicht

2

Representational State Transfer (abgekürzt **REST**, seltener auch **ReST**) bezeichnet ein [Programmierparadigma](#) für [verteilte Systeme](#), insbesondere für [Webservices](#).

REST ist eine Abstraktion der Struktur und des Verhaltens des [World Wide Web](#). REST hat das Ziel, einen Architekturstil zu schaffen, der die Anforderungen des modernen Web besser darstellt.

Dabei unterscheidet sich REST vor allem in der Forderung nach einer einheitlichen Schnittstelle (siehe Abschnitt [Prinzipien](#)) von anderen Architekturstilen.

REST stellt eine einfache Alternative zu ähnlichen Verfahren wie [SOAP](#) und [WSDL](#) und dem verwandten Verfahren [RPC](#) dar.

Anders als bei vielen verwandten [Architekturen](#) kodiert REST keine Methodeninformation in den [URI](#), da der URI Ort und Namen der Ressource angibt, nicht aber die Funktionalität, die der Web-Dienst zu der Ressource anbietet.

Der Vorteil von REST liegt darin, dass im WWW bereits ein Großteil der für REST nötigen Infrastruktur (z. B. Web- und Application-Server, [HTTP](#)-fähige Clients,

Dienste per se REST-konform sind. Eine Ressource kann dabei über verschiedene [Medientypen](#) dargestellt werden, auch *Repräsentation der Ressource* genannt.

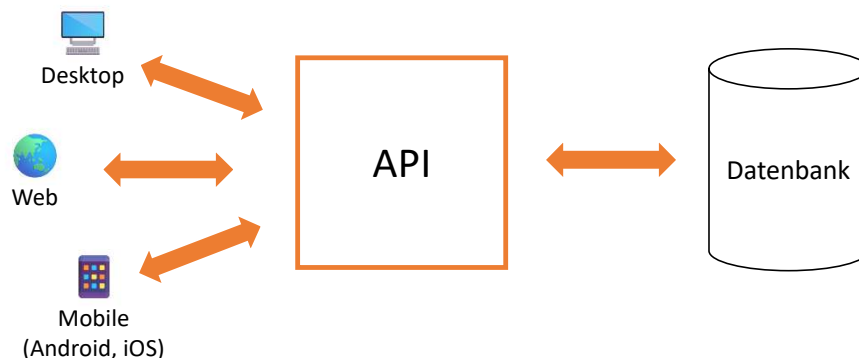
Das REST-Paradigma entwickelte sich aus dem 1994 von [Roy Fielding](#) entworfenen HTTP Object Model.

Fielding entwickelte seine Idee von einem einheitlichen Konzept über die Jahre weiter, bis er 2000 den REST-Architekturstil im Rahmen seiner [Dissertation](#) veröffentlichte.^[2] Das Programmierparadigma der „RESTful Application“ wurde allerdings häufig falsch umgesetzt und findet eigentlich erst seit neuestem (Stand 2014) Anklang in der Welt des World Wide Web. In seiner Arbeit geht Fielding dabei auf die verschiedenen Anforderungen ein, die für die Webarchitektur wichtig sind.

Theorie - RESTful Services

Wozu REST?

- Eine Einheitliche Schnittstelle für mehrere Konsumenten anbieten



3

Representational State Transfer (abgekürzt **REST**, seltener auch **ReST**) bezeichnet ein [Programmierparadigma](#) für [verteilte Systeme](#), insbesondere für [Webservices](#).

REST ist eine Abstraktion der Struktur und des Verhaltens des [World Wide Web](#). REST hat das Ziel, einen Architekturstil zu schaffen, der die Anforderungen des modernen Web besser darstellt.

Dabei unterscheidet sich REST vor allem in der Forderung nach einer einheitlichen Schnittstelle (siehe Abschnitt [Prinzipien](#)) von anderen Architekturstilen.

REST stellt eine einfache Alternative zu ähnlichen Verfahren wie [SOAP](#) und [WSDL](#) und dem verwandten Verfahren [RPC](#) dar.

Anders als bei vielen verwandten [Architekturen](#) kodiert REST keine Methodeninformation in den [URI](#), da der URI Ort und Namen der Ressource angibt, nicht aber die Funktionalität, die der Web-Dienst zu der Ressource anbietet.

Der Vorteil von REST liegt darin, dass im WWW bereits ein Großteil der für REST nötigen Infrastruktur (z. B. Web- und Application-Server, [HTTP](#)-fähige Clients,

nötigen Infrastruktur (z. B. Web- und Application-Server, [HTTP](#)-fähige Clients, [HTML](#)- und [XML](#)-Parser, Sicherheitsmechanismen) vorhanden ist, und viele Web-Dienste per se REST-konform sind. Eine Ressource kann dabei über verschiedene [Medientypen](#) dargestellt werden, auch *Repräsentation der Ressource* genannt.

Das REST-Paradigma entwickelte sich aus dem 1994 von [Roy Fielding](#) entworfenen HTTP Object Model.

Fielding entwickelte seine Idee von einem einheitlichen Konzept über die Jahre weiter, bis er 2000 den REST-Architekturstil im Rahmen seiner [Dissertation](#) veröffentlichte.^[2] Das Programmierparadigma der „RESTful Application“ wurde allerdings häufig falsch umgesetzt und findet eigentlich erst seit neuestem (Stand 2014) Anklang in der Welt des World Wide Web. In seiner Arbeit geht Fielding dabei auf die verschiedenen Anforderungen ein, die für die Webarchitektur wichtig sind.

Theorie - RESTful Services

Welche Vorteile bietet REST?

- Vermeidung mehrfacher Logik für jede Anwendung
- Code ist weniger fehleranfällig, da logische Trennung forciert wird
- Nicht alle Front-End-Frameworks können mit einer Datenbank kommunizieren
- Ermöglicht bessere Wartbarkeit, Skalierbarkeit und Testbarkeit
- Bessere Sicherheit, da direkter Zugriff auf die Datenbank vermieden wird

4

Representational State Transfer (abgekürzt **REST**, seltener auch **ReST**) bezeichnet ein [Programmierparadigma](#) für [verteilte Systeme](#), insbesondere für [Webservices](#).

REST ist eine Abstraktion der Struktur und des Verhaltens des [World Wide Web](#). REST hat das Ziel, einen Architekturstil zu schaffen, der die Anforderungen des modernen Web besser darstellt.

Dabei unterscheidet sich REST vor allem in der Forderung nach einer einheitlichen Schnittstelle (siehe Abschnitt [Prinzipien](#)) von anderen Architekturstilen.

REST stellt eine einfache Alternative zu ähnlichen Verfahren wie [SOAP](#) und [WSDL](#) und dem verwandten Verfahren [RPC](#) dar.

Anders als bei vielen verwandten [Architekturen](#) kodiert REST keine Methodeninformation in den [URI](#), da der URI Ort und Namen der Ressource angibt, nicht aber die Funktionalität, die der Web-Dienst zu der Ressource anbietet.

Der Vorteil von REST liegt darin, dass im WWW bereits ein Großteil der für REST nötigen Infrastruktur (z. B. Web- und Application-Server, [HTTP](#)-fähige Clients,

[HTML](#)- und [XML](#)-Parser, Sicherheitsmechanismen) vorhanden ist, und viele Web-Dienste per se REST-konform sind. Eine Ressource kann dabei über verschiedene [Medientypen](#) dargestellt werden, auch *Repräsentation der Ressource* genannt.

Das REST-Paradigma entwickelte sich aus dem 1994 von [Roy Fielding](#) entworfenen HTTP Object Model.

Fielding entwickelte seine Idee von einem einheitlichen Konzept über die Jahre weiter, bis er 2000 den REST-Architekturstil im Rahmen seiner [Dissertation](#) veröffentlichte.^[2] Das Programmierparadigma der „RESTful Application“ wurde allerdings häufig falsch umgesetzt und findet eigentlich erst seit neuestem (Stand 2014) Anklang in der Welt des World Wide Web. In seiner Arbeit geht Fielding dabei auf die verschiedenen Anforderungen ein, die für die Webarchitektur wichtig sind.

Theorie - RESTful Services

Welche Eigenschaften muss ein RESTful Service haben?

- **Client Server Architektur**
Eine einheitliche Schnittstelle trennt Client und Server.
- **Statelessness**
Zustandslose Interaktion zwischen Client und Server.
- **Cacheability**
Nachrichten vom Server (Responses) müssen sich selbst als cacheable oder nicht cacheable auszeichnen, damit der Client keine veralteten Daten zwischenspeichert.

5

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Theorie - RESTful Services

Welche Eigenschaften muss ein RESTful Service haben?

- **Layered System**
Der Client hat keine Information darüber, ob er direkt mit dem finalen Server verbunden ist.
- **Uniform interface**
Jeder Dienst, jede Methode oder jeder Status besitzt eine eindeutige Adresse.

6

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Theorie - RESTful Services

Was ist eine Web-API?

- API: Application Programming Interface
- Ermöglicht Datenaustausch zwischen Front- und Backend
- Web-API ist eine spezielle API für das Internet oder Netzwerk
- Ein Satz von Regeln und Protokollen stellt die Grundlage dar
- HTTP wird als Kommunikationsprotokoll verwendet

Theorie - RESTful Services

Schlüsselemente von Web-APIs

- **Mehrfache Repräsentationen einer Ressource**
Jeder Service kann verschiedene „Repräsentationen“ einer Datenquelle liefern, z. B. JSON oder XML.
- **Uniform Resource Identifier (URI)**
Lokalisiert eine spezifische Ressource über eine Zugriffsmethode (z. B. HTTP) in einem Netzwerk.
- **Uniform Interface**
Service verwendet HTTP-Verben, um auf adressierbare Ressourcen über eine URI zugreifen zu können.

8

Adressierbarkeit von Ressourcen

Jede Information, die über einen URI kenntlich gemacht wurde, wird als Ressource gekennzeichnet. Jeder REST-konforme Dienst hat eine eindeutige Adresse, den [Uniform Resource Locator](#) (URL). Diese „Straße und Hausnummer im Netz“ standardisiert den Zugriffsweg zum Angebot eines Webservices für eine Vielzahl von Anwendungen (Clients). Eine konsistente Adressierbarkeit erleichtert es außerdem, einen Webservice als Teil eines [Mashups](#) weiterzuverwenden.

Einheitliche Schnittstelle

Dies ist das Hauptunterscheidungsmerkmal von allen weiteren Architekturstilen. Dabei besteht diese aus vier weiteren Eigenschaften. Ziel ist die Einheitlichkeit der Schnittstelle und somit ihre einfache Nutzung.

Repräsentationen zur Veränderung von Ressourcen

Die unter einer Adresse zugänglichen Dienste können unterschiedliche Darstellungsformen (Repräsentationen) haben. Ein REST-konformer Server kann je nachdem, was die Anwendung anfordert, verschiedene Repräsentationen einer Ressource ausliefern, z. B. in verschiedenen Sprachen oder Formaten ([HTML](#), [JSON](#) oder [XML](#)) oder auch die Beschreibung oder Dokumentation des Dienstes. Die Veränderung einer Ressource (also deren aktuellen Status) soll nur über eine Repräsentation erfolgen.

Selbstbeschreibende Nachrichten

REST-Nachrichten sollen selbstbeschreibend sein. Dazu zählt u. a. die Verwendung von Standardmethoden. Über diese Standardmethoden lassen sich Ressourcen manipulieren. Als Beispiel seien an dieser Stelle die HTTP-Verben genannt; Details siehe unten.

Mehrschichtige Systeme

Die Systeme sollen mehrschichtig aufgebaut sein. Dadurch reicht es, dem Anwender lediglich eine Schnittstelle anzubieten. Dahinterliegende Ebenen können verborgen bleiben und somit die Architektur insgesamt vereinfacht werden.

Code on Demand (optional)

Unter Code on Demand ist zu verstehen, dass erst im Bedarfsfall an den Client Code zur lokalen Ausführung übertragen werden kann.

Ein Beispiel hierfür wäre die Übertragung von JavaScript-Code bei einer HTML-Repräsentation. Diese Forderung von Fielding ist optional.

Theorie - RESTful Services

Schlüsselemente von Web-APIs

- **Zustandslose Kommunikation (stateless)**
Client muss Informationen im Request zur Verfügung stellen, damit Server eine entsprechende Response liefern kann.
- **Caching**
Response vom Server kann aus dem Cache abgerufen werden.
Aber: Eine Anfrage, die nicht gestellt werden muss, ist die schnellste Anfrage.
- **Links**
Server stellt bearbeitbare Ressource mit Hilfe eines HATEOS Links zur Verfügung.
Konzept: HATEOAS (Hypermedia as the Engine of Application State Source)

9

Zustandslosigkeit

Jede REST-Nachricht enthält alle Informationen, die für den Server bzw. Client notwendig sind, um die Nachricht zu verstehen. Weder der Server noch die Anwendung soll Zustandsinformationen zwischen zwei Nachrichten speichern. Man spricht daher von einem [zustandslosen](#) (englisch: *stateless*) Protokoll. Jede Anfrage eines Clients an den Server ist insofern in sich geschlossen, als sie sämtliche Informationen über den Anwendungszustand beinhaltet, die vom Server für die Verarbeitung der Anfrage benötigt werden.

Zustandslosigkeit in der hier beschriebenen Form begünstigt die [Skalierbarkeit](#) eines Webservices. Beispielsweise können eingehende Anfragen im Zuge der [Lastverteilung](#) unkompliziert auf beliebige Maschinen verteilt werden: Da jede Anfrage in sich geschlossen ist und Anwendungsinformationen somit ausschließlich auf der Seite des Clients vorgehalten werden, ist auf der Seite des Servers keine Sitzungsverwaltung erforderlich. In der Praxis nutzen deswegen viele HTTP-basierte Anwendungen [Cookies](#) und andere Techniken, um Zustandsinformationen auf der Server-Seite zu behalten. Weiterhin begünstigt wird die Ausfallsicherheit, weil die Zustandslosigkeit fordert, dass transaktionale Datenübertragung in einem einzigen Seitenaufruf erfolgt.

Client-Server

Es gilt generell die Anforderung, dass alle Eigenschaften der [Client-Server-Architektur](#) gelten. Dabei stellt der Server einen Dienst bereit, der bei Bedarf vom Client angefragt werden kann.

Caching

[HTTP Caching](#) soll genutzt werden, wobei aber gilt: Eine Anfrage, die nicht gestellt werden muss, ist die schnellste Anfrage.

„Hypermedia as the Engine of Application State“ (HATEOAS)

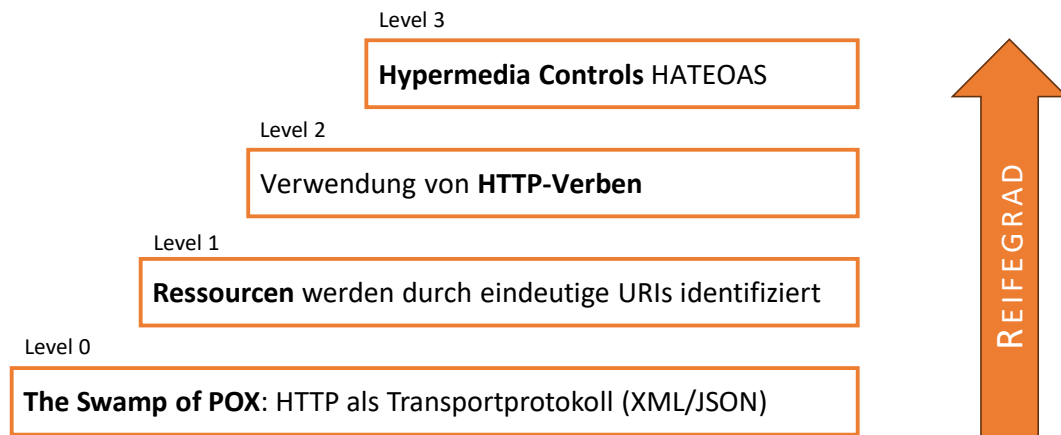
HATEOAS bedeutet, dass ein REST-Client sich ausschließlich durch das Folgen von URIs (Uniform Resource Identifier) im Hypermedia-Format durch die Webanwendung bewegen kann. Das bedeutet, dass der Client, abgesehen von einem grundsätzlichen Verständnis von Hypermedia, keine weiteren Informationen benötigt, um mit der Applikation bzw. dem Server zu interagieren

HATEOAS und Web-APIs

In der Praxis ermöglicht HATEOAS es, dass die Schnittstelle eines REST-Services jederzeit angepasst werden kann, ohne dass der Client geändert werden muss. Dies wird durch die Bereitstellung von Hypermedia-Links in den API-Antworten erreicht, die dem Client ermöglichen, dynamisch durch die verschiedenen Ressourcen und Operationen der API zu navigieren. Durch die Implementierung von HATEOAS wird die API selbstbeschreibend und erleichtert die Interaktion zwischen Client und Server, ohne dass der Client die Struktur der API im Voraus kennen muss

Theorie - RESTful Services

Richardson Maturity Model (RMM)



Die 4 Level der Einhaltung und Konformität mit REST-Prinzipien

10

IN-COURSE REMARKS

- HATEOAS: Hypermedia as the Engine of Application State
- Client interagiert mit einer Anwendung im Netzwerk nur durch dynamisches Hypermedia
- REST Client benötigt kein Wissen darüber, wie mit einer Anwendung interagiert werden soll

Das Richardson Maturity Model hilft Entwicklern und Architekten, den Reifegrad ihrer Web-Services zu bewerten und zu verbessern, indem es eine klare Struktur und Zielvorgaben für die Einhaltung von REST-Prinzipien bietet³.

Das Richardson Maturity Model (RMM) ist ein Reifegradmodell, das von Leonard Richardson entwickelt wurde, um Web-APIs basierend auf ihrer Einhaltung und Konformität mit REST-Prinzipien zu klassifizieren¹. Es unterteilt die Reife von Web-Services in vier Ebenen, die jeweils unterschiedliche Aspekte der REST-Architektur berücksichtigen².

1. Level 0 - The Swamp of POX: (Plain Old XML)

Auf dieser Ebene werden HTTP als Transportprotokoll und XML oder JSON als Datenformat verwendet, aber es gibt keine Nutzung von Ressourcen oder HTTP-Methoden. Dies ist die grundlegendste Form der Web-Services, bei der alle Anfragen an eine einzige URI gesendet werden und nur eine HTTP-Methode (meist POST) verwendet wird².

2. Level 1 - Resources:

Hier werden Ressourcen durch eindeutige URIs identifiziert. Jede Ressource hat eine eigene URI, aber es werden immer noch hauptsächlich nur eine HTTP-Methode (meist POST) verwendet. [Dies verbessert die Struktur im Vergleich zu Level 0, aber es fehlen noch die vollständigen REST-Prinzipien².](#)

1. Level 2 - HTTP Verbs:

Auf dieser Ebene werden neben eindeutigen URIs auch verschiedene HTTP-Methoden (GET, POST, PUT, DELETE) verwendet, um unterschiedliche Operationen auf den Ressourcen durchzuführen. [Dies ermöglicht eine klarere und standardisierte Interaktion mit den Ressourcen².](#)

2. Level 3 - Hypermedia Controls (HATEOAS):

Dies ist die höchste Ebene der REST-Reife. Neben der Nutzung von URIs und HTTP-Methoden werden hier auch Hypermedia-Links in den Antworten bereitgestellt, die es dem Client ermöglichen, dynamisch durch die API zu navigieren. [Dies macht die API selbstbeschreibend und erleichtert die Interaktion zwischen Client und Server³.](#)

Theorie - RESTful Services

REST Operationen mit Hilfe von HTTP-Verben (Methoden)

Methode	CRUD action	Beschreibung
GET	Read	Gibt einen einzelnen oder eine Sammlung von Einträgen zurück.
POST	Create	Erzeugen eines neuen Eintrags.
PUT	Update	Update (Aktualisierung) eines bestehenden Eintrags.
DELETE	Delete	Löschen eines Eintrags.
PATCH	Update	Teile eines Eintrags werden geändert / aktualisiert
Head	N/A	Liest die Message Header anhand der URI aus

11

1.)

POST /groups/{group id}/activation Creates (or requests the creation of) an activation.

PATCH /groups/{group id}/activation Updates some details of an existing activation. Since a group has only one activation, we know what activation-resource we are referring to.

PUT /groups/{group id}/activation Inserts-or-replaces the old activation. Since a group has only one activation, we know what activation-resource we are referring to.

DELETE /groups/{group id}/activation Will cancel, or remove the activation.

2.) Referenz

https://www.w3schools.com/tags/ref_httpmethods.asp

<https://tools.ietf.org/html/rfc5789>

Theorie - RESTful Services

Die wichtigsten [HTTP-Statuscodes](#)

Code	Beschreibung	Beispiele
1xx	Informativ	100 Continue
2xx	Erfolgreich	200 OK, 204 No Content
3xx	Umleitung	303 See Other
4xx	Client Error	400 Bad Request, 403 Forbidden, 404 Not Found
5xx	Server Error	500 Internal Server Error

Theorie - RESTful Services

Original Dissertation über REST



<http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

13

IN-COURSE REMARKS

g
g

PREPARATION REMARKS

g
g

Theorie - RESTful Services

Wichtige Merkmale von Web-APIs

- HTTP-basierte Kommunikation
 - HTTP-Verben (Methoden)
 - API-Endpunkte als URLs
- Datenaustauschformate (XML oder JSON)
- RESTful-Architektur
 - Zustandslos und Standardisierte HTTP-Methoden
 - Ressourcen hierarchisch mit eindeutigen URLs
- Authentifizierung und Autorisierung
 - Sicherheitsmechanismen um Zugriff für autorisierte Clients zu gewähren
 - Gängige Authentifizierungsmethoden: API-Schlüssel, OAuth und JWT (JSON Web Tokens)

Zusammengefasst lässt sich sagen... s. o.

Unterschiede zwischen REST- und SOAP-Diensten

- SOAP: Simple Object Access Protocol
- XML-basiertes Protokoll
- zustandslos oder zustandsvoll
- Erzwingt XML
- SOAP-Nachricht besteht aus Umschlag mit Headern und Body
- Operationsbasierte Dienste
- Starre Struktur und wegen XML-Format langsamer
- REST: Representational State Transfer
- Architekturmuster welches HTTP nutzt
- völlig zustandslos
- XML oder JSON
- Nutzt HTTP-Header und HTTP-Methoden
- Ressourcenbasierte Dienste via URIs
- REST ist flexibler, vielseitiger und performanter

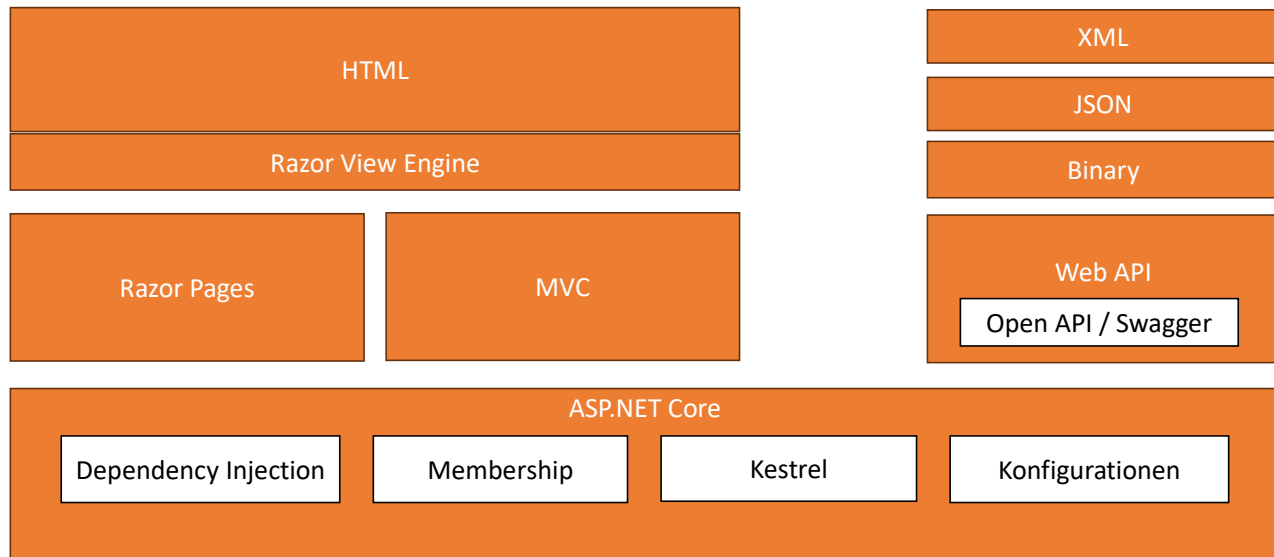
> allgemein einfacher handhabbar

ASP.NET Core – Web API

Das Microsoft Framework zur Erstellung von REST Services

ASP.NET Core – Web API

Web API ist ein Teil von ASP.NET Core



17

- Kestrel ist ein leichtgewichtiger Web server in ASP.NET Core (crossplatform)
- Im Gegensatz dazu .NET Framework Web API nur IIS auf Windows

Grundlagen ASP.NET Web API

Was ist die ASP.NET Core Web API ?

- **Eine Bibliothek zur Erstellung von HTTP Services**
 - HTTP ermöglicht eine hohe Skalierbarkeit
 - Laut Microsoft ein Leichtgewicht
- **Entwickelt, um REST Services leicht zu erstellen**
- **Perfekt für heterogene Gerät/OS Szenarien**
 - Keine Probleme im Bezug auf Versionierung wie bei SOAP
- **Mehrwert für ASP.NET Core:**
 - **Razor Pages**
 - **MVC**
 - **Blazor (Server, WebAssembly)**

18

IN-COURSE REMARKS

- Vorteil Selfhosting: Nicht auf Web Anwendungen limitiert
- Wie MVC: Controller, Routing, Action Filter, Formatter, ModelBinder, Self Hosting, Open Source

PREPARATION REMARKS

g

g

Grundlagen ASP.NET Web API

Was kann die Web API?

- **Keine Bindung an Webanwendungen**
- **Konvention über Konfiguration**
 - Verwendung von HTTP-Methoden und HTTP Status Codes
- **Content Negotiation**
 - „Inhaltsvereinbarung“ zwischen Client-Server
 - Die Web API gibt das geforderte Ergebnis serialisiert in der Form aus, die vom Client als akzeptierte Form angegeben wird (XML, JSON, Binär...).
 - Steuerung über den Request Header: Client sagt was er versteht
- **Caching**
 - Unterstützt Standard HTTP Caching Mechanismus

19

IN-COURSE REMARKS

g

g

PREPARATION REMARKS

g

g

Grundlagen ASP.NET Web API

Wo kann die ASP.NET Web API verwendet werden?

- Backend für Mobile Clients
- Backend für Single Page Applikation
- Backend für Desktop Anwendungen
- Ergänzt ASP.NET Core Anwendungen (Razor Pages / MVC)
- Als Azure-Dienst gehostet werden

20

IN-COURSE REMARKS

- Gutes Backend für Apps: Android mit Java, iOS mit SWIFT oder WinPhone mit C#
- SPA Apps mit AngularJS
- Desktop: Informationen nach Außen geben

g

g

PREPARATION REMARKS

g

g

System Requirements

- Betriebssystem: Windows, Mac, Linux
- Microsoft .NET SDK 8
- Editor / IDE
 - [Visual Studio 2022](#) (z. B. Community Edition)
 - Visual Studio Code + .Net Core-CLI
- API-Client (Browser, Fiddler, Postman, Hopscotch, ...)

```
C:\Users\oc1>dotnet
Usage: dotnet [options]
Usage: dotnet [path-to-application]

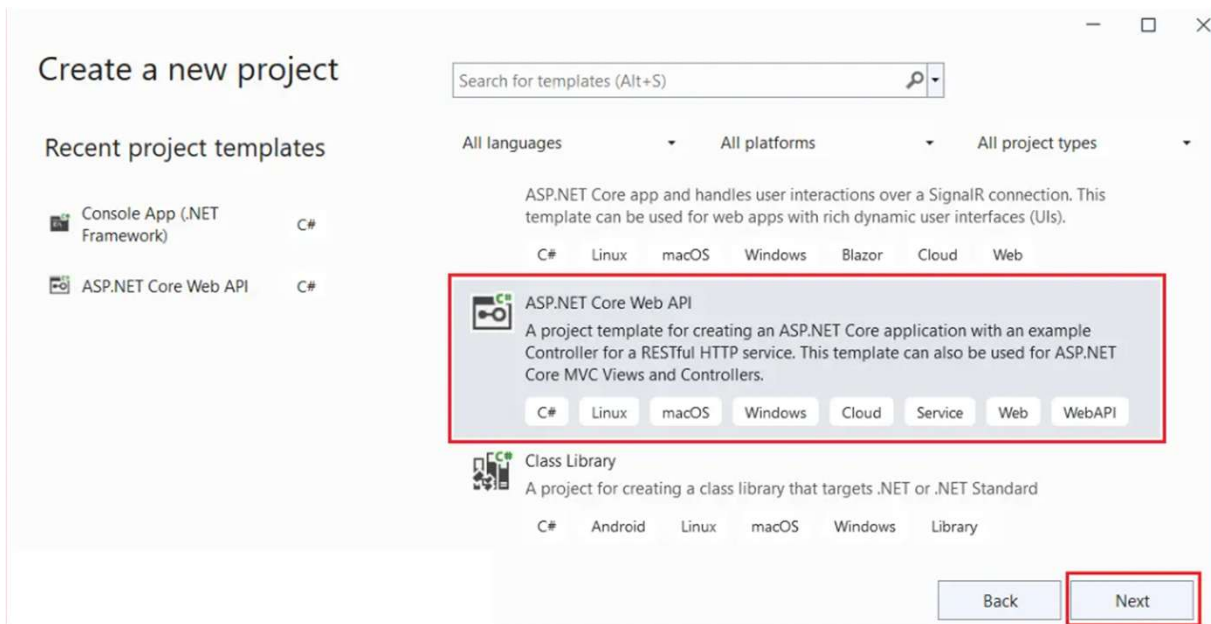
Options:
  -h|--help           Display help.
  --info              Display .NET information.
  --list-sdks          Display the installed SDKs.
  --list-runtimes      Display the installed runtimes.

path-to-application:
  The path to an application .dll file to execute.

C:\Users\oc1>
```

Demo

Neues Projekt erstellen



Neues Projekt erstellen

Einrichten des ASP.NET Core Web API Projekts

Additional information

ASP.NET Core Web API C# Linux macOS Windows API Cloud Service Web Web API

Framework 

.NET 8.0 (Long Term Support)

Authentication type 

None

☒ Configure for HTTPS 

☐ Enable Docker 

Docker OS 

Linux

☒ Enable OpenAPI support 

☒ Do not use top-level statements 

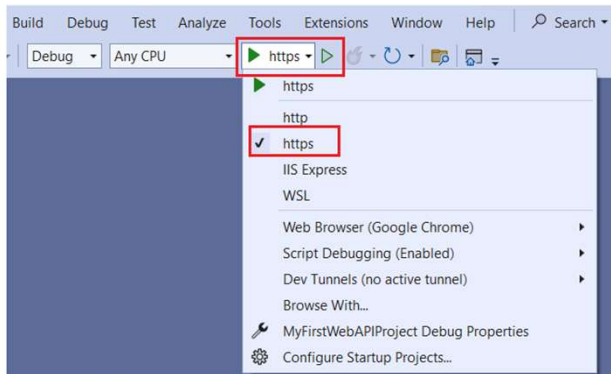
☒ Use controllers 

Back Create

- **Für HTTPS konfigurieren:** VS richtet erforderliche SSL-Zertifikate ein
- **Docker-Unterstützung:** Fügt Docker-Datei hinzu, wodurch App problemlos als Container verpackt werden kann
- **OpenAPI-Unterstützung:** Richtet Swagger Konfiguration automatisch ein. Dadurch wird eine automatische Dokumentation und UI zum Testen der API-Methoden generiert.
- **Controller verwenden:** Erstellt Controller statt eine Minimal API zu verwenden

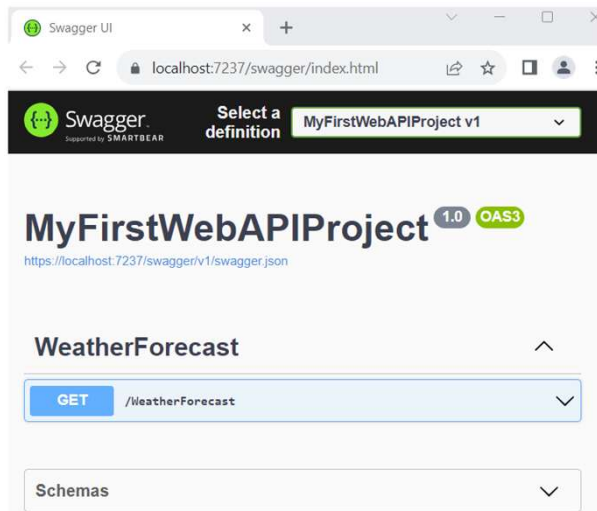
Projekt bauen und ausführen

- Solution mit **Strg+Shift+B** bauen



- **Als http starten**
- **Als https starten** (evtl. muss bei der ersten Verwendung dem System ein lokales, unsigniertes Zertifikat hinzugefügt werden)
- **IIS Express** ist eine für Entwickler optimierte, leichte, in sich geschlossene Version von IIS und existiert seit dem .Net Framework.
- **WSL** (Windows-Subsystem für Linux) ermöglicht Entwicklern .NET-Apps in einer Linux-Umgebung zu testen und auszuführen

Projekt bauen und ausführen



- Swagger Seite öffnet sich im Browser
- Die Portnummer kann abweichen

- <https://localhost:7237/WeatherForecast> aufrufen

```
localhost:7237/WeatherForecast  
[{"date": "2024-01-31", "temperatureC": 22, "temperatureF": 71, "summary": "Warm"},  
{"date": "2024-02-01", "temperatureC": 25, "temperatureF": 76, "summary": "Hot"},  
{"date": "2024-02-02", "temperatureC": -5, "temperatureF": 24, "summary": "Chilly"},  
{"date": "2024-02-03", "temperatureC": -1, "temperatureF": 31, "summary": "Chilly"},  
{"date": "2024-02-04", "temperatureC": -11, "temperatureF": 13, "summary": "Balmy"}]
```

25

<https://dotnettutorials.net/lesson/swagger-api-in-asp-net-core-web-api/>

Projekt testen

The image shows the Swagger UI for the `GET /WeatherForecast` endpoint. The `Parameters` tab is active, showing "No parameters". A "Try it out" button is visible. Below, the `Responses` section shows a 200 status code with a "Success" description. A dropdown menu for "Media type" is set to "text/plain". An "Example Value" is displayed as a JSON object:

```
{
  "date": "2024-01-31",
  "temperatureC": 0,
  "temperatureF": 0,
  "summary": "string"
}
```

The image shows the Swagger UI for the `GET /WeatherForecast` endpoint. The `Parameters` tab is active, showing "No parameters". A red arrow points to the "Execute" button with the text "Click Here to Execute the API". Below, the `Responses` section shows a 200 status code with a "Server response" tab. The "Response body" is displayed as a JSON array of two objects:

```
[
  {
    "date": "2024-02-01",
    "temperatureC": 22,
    "temperatureF": 71,
    "summary": "Cool"
  },
  {
    "date": "2024-02-02",
    "temperatureC": 91,
    "temperatureF": 123,
    "summary": "Sweating"
  }
]
```

26

Anpassen von Swagger in .NET 8

Durch Anpassen von Swagger in .NET 8 können Sie die Swagger-Benutzeroberfläche und die generierte OpenAPI-Spezifikation besser an die Anforderungen Ihres Projekts anpassen. Dies kann benutzerdefinierte Informationen wie API-Beschreibungen und Kontaktinformationen, Lizenzen, Versionen usw. umfassen.

Sie können die Swagger-Metadaten anpassen, indem Sie die SwaggerGen-Optionen in Program.cs konfigurieren. Ändern Sie also den AddSwaggerGen-Dienst wie folgt. Wie Sie hier sehen können, haben wir weitere Informationen zu unseren APIs bereitgestellt, wie Titel, Version, Beschreibung, Servicebedingungen, Kontakt und Lizenz.